The Land Acquisition Workflow

A proprietary P2P/Escrow interface

bridging blockchain logic with real-world legal milestones for land purchases

Document Version 1.0
March 26, 2026

System Architecture

Core Components

1. **MultiSigWallet.sol** - Base multi-signature wallet implementation
2. **NotaryMediatedEscrow.sol** - Main escrow contract with milestone-based releases
3. **EscrowFactory.sol** - Factory for modular escrow deployment
4. **Frontend Interface** - React-based 4-phase workflow UI
5. **Document Hashing System** - IPFS + on-chain verification

Key Features

- **Multi-Sig Security**: Requires multiple confirmations for fund releases
- **Notary Oracle**: Legal Notary verifies legal milestones before fund releases
- **Document Hashing**: All legal documents hashed on-chain for verification
- **4-Phase Workflow**: Discovery → Safety Gate → Execution → Final Settlement
- **Escrow Factory**: Modular deployment for any land purchase

Quick Start

Prerequisites

- Node.js >= 16.0.0
- Hardhat development environment
- MetaMask or compatible Web3 wallet

Installation

```
# Clone the repository
git clone <repository-url>
cd multisig

# Install dependencies
npm install

# Compile contracts
npx hardhat compile

# Run tests
npx hardhat test
```

```
# Deploy contracts
npx hardhat run scripts/deploy.js --network localhost
```

Frontend Setup

```
# Navigate to frontend directory
cd frontend

# Install dependencies
npm install

# Start development server
npm start
```

4-Phase Workflow

Phase 1: Discovery & Lock-up

- **User Action**: Client deposits total budget (e.g., $5,000) into Treasury
- **System Action**: Funds are locked in multi-sig
- **UI Component**: Curated list of "Digital Proofs" (Geo-tagged photos/videos)

Phase 2: Safety Gate ($200 Due Diligence)

- **Condition**: First milestone of $200 unlocked for plot selection
- **Backend Logic**: Goes to Legal Notary's wallet for Registry Search and site visit
- **UI Component**: "Document Vault" for viewing Registry Search and Site Visit Report

Phase 3: Execution (Commitment Milestone)

- **Condition**: Both parties sign draft agreement on-platform
- **On-Platform Signing**: Client or seller uploads draft, both parties sign with real-time notifications
- **Notary Role**: Legal Notary verifies signed documents and clicks "Proceed to Next Step"
- **Smart Contract Action**: Release percentage-based deposit to seller after notarization
- **UI Component**: Document signing system with party status tracking

Phase 4: Final Settlement & Notary Release

- **Condition**: Legal Notary confirms Original Title and LCB Consents
- **Smart Contract Action**: Release balance to seller + facilitation fee.
- **UI Component**: "Finalization Status" showing transfer progress

Smart Contract Details

NotaryMediatedEscrow.sol - Key Functions

```
// Create new escrow
function createEscrow(address _seller, address _notary, uint _totalAmount) returns (u

// Fund escrow with total amount
function fundEscrow(uint _escrowId) payable

// Upload draft sale agreement (client or seller)
function uploadDraftAgreement(uint _escrowId, bytes32 _hash)

// Sign document on platform (both parties)
function signDocument(uint _escrowId, DocumentType _docType)

// Notarize signed document and proceed (notary only)
function notarizeSignedDocument(uint _escrowId, DocumentType _docType)

// Upload other documents (notary only)
function uploadDocument(uint _escrowId, DocumentType _docType, bytes32 _hash)

// Finalize transfer (notary only)
function finalizeTransfer(uint _escrowId)
```

Milestone Structure

```
enum Milestone {
    DueDiligence,    // $200 for registry search and site visit
    Deposit,         // Percentage-based deposit to seller
    FinalSettlement  // Balance + facilitation fee
}
```

Document Types

```
enum DocumentType {
    RegistrySearch,
    SiteVisitReport,
    DraftSaleAgreement,
    SignedSaleAgreement,
    OriginalTitle,
```

```
    LCBConsent
}

enum SigningStatus {
    NotStarted,
    Draft,
    ClientSigned,
    SellerSigned,
    BothSigned,
    NotaryVerified
}
```

EscrowFactory.sol - Key Functions

```
// Create escrow with default notary
function createEscrowWithNotary(address _seller, uint _totalAmount) returns (address)

// Create escrow with custom notary
function createEscrow(address _seller, address _notary, uint _totalAmount) returns (a

// Batch create multiple escrows
function createBatchEscrows(address[] memory _sellers, address[] memory _notaries, ui
```

Frontend Components

Main Interface

- **EscrowInterface.jsx** - Main dashboard with phase navigation
- **ProgressBar.jsx** - Real-time progress tracking
- **MultiSigVerification.jsx** - Transaction confirmation interface

Phase Components

- **DiscoveryPhase.jsx** - Property selection and fund locking
- **SafetyGatePhase.jsx** - Due diligence document verification
- **ExecutionPhase.jsx** - Sale agreement confirmation and deposit release
- **FinalSettlementPhase.jsx** - Title verification and final settlement

Security Features

Multi-Sig Verification

- All fund releases require multiple confirmations

- Configurable number of required signatures
- Transaction queue with status tracking

Document Hashing

- SHA-256 hashing of all legal documents
- On-chain storage of document hashes
- IPFS storage for original documents
- Public verification capability

Condition Hooks

- Progress bar with real-time status updates
- Multi-sig verification for fund releases
- Document hashing verification before payments
- State-based access control

Financial Flow

Fee Structure

```
Total Purchase Amount: $5,000
 Due Diligence Fee: $200 (4%)
 Deposit (10%): $500
 Final Balance: $4,200
 Facilitation Fee: $100 (2% of total)
```

Release Schedule

1. $200 → Legal Notary (Due Diligence)
2. $500 → Seller (Deposit after Sale Agreement)
3. $4,200 → Seller (Final Balance after Title Transfer)
4. $100 → Facilitation Fee

Development

Contract Deployment

```
# Start local Hardhat network
npx hardhat node

# Deploy contracts (in separate terminal)
npx hardhat run scripts/deploy.js --network localhost
```

Frontend Development

```
cd frontend
npm install
npm start
```

Testing

```
# Run contract tests
npx hardhat test

# Run frontend tests
cd frontend
npm test
```

Configuration

Environment Variables

```
# Network Configuration
NETWORK_ID=1337
RPC_URL=http://localhost:8545

# Contract Addresses (after deployment)
MULTISIG_WALLET_ADDRESS=0x...
NOTARY_MEDIATED_ESCROW_ADDRESS=0x...
ESCROW_FACTORY_ADDRESS=0x...

# IPFS Configuration
IPFS_GATEWAY=https://ipfs.io/ipfs/

# Notary Configuration
LEGAL_NOTARY_ADDRESS=0xNotary12345678901234567890123456789012345678901234567890
TREASURY_ADDRESS=0xTreasury123456789012345678901234567890123456789012345678
```

Integration Guide

Backend Integration

1. Deploy contracts to desired network
2. Configure EscrowFactory with Legal Notary's address
3. Set up multi-sig owners
4. Fund Treasury with initial capital if needed

Frontend Integration

1. Update contract addresses in App.jsx
2. Configure Web3 provider for target network
3. Set up IPFS gateway for document storage
4. Test wallet integration with MetaMask

Legal Integration

1. Onboarding authorized notary
2. Establish document templates for each phase
3. Set up verification process for legal documents
4. Create dispute resolution mechanism

Important Notes

Security Considerations

- Always verify contract addresses before deployment
- Test thoroughly on testnet before mainnet deployment
- Ensure proper multi-sig configuration
- Regular security audits recommended

Legal Compliance

- Ensure proper licensing for notary services
- Comply with local land transfer regulations
- Maintain proper audit trails
- Consider jurisdiction-specific requirements

Operational Requirements

- Proper notary certification
- Treasury must be properly secured
- Document verification process must be established
- Customer support infrastructure needed

Support

For technical support or questions:

- Create an issue in the repository
- Contact the development team
- Review documentation and test cases

© 2026 Escrow Secure Land Purchase Platform
Powered by Ethereum Smart Contracts & Notary Verification